

AI Engineer (Data): Practical Lab Guide

Student edition: Azure Databricks, Labs 1 to 12

Novel Vista × ITC Infotech | 48-hour, 6-week programme

Version 2.0, September 2026 | Lab pack: freshcart-labs version 2

All twelve labs tested end to end on Azure Databricks serverless compute (Python 3.12.3) on 24 September 2026, and on the NovelVista Windows lab machine (Python 3.14.7) on 16 September 2026.

Contents

Contents

1

1. How to use this guide

3

1.1 Conventions

4

1.2 Which labs are in which week

4

1.3 Between-session assignments

4

2. Set up

6

2.1 What you need

6

2.2 Sign in

6

2.3 Make your own copy of the notebooks

6

2.4 Connect a notebook to a machine

6

2.5 The first two cells of every lab

6

2.6 How to run a lab

7

2.7 Where everything lives

7

3. Week 1: Embeddings and vector stores

8

Lab 1: Ingest five formats into one indexed corpus

8

Lab 2: Benchmark three vector stores and pick one

10

4. Week 2: Retrieval and grounding

13

Lab 3: Hybrid retrieval, reranking and the lift

13

Lab 4: A RAG API with verifiable citations

14

5. Week 3: NL2SQL and semantic layers

15

Lab 5: NL2SQL on the star schema, with an accuracy number

15

Lab 6: Natural language to insight, with a semantic layer

15

6. Week 4: Data quality and governance

18

Lab 7: Trace a wrong number, then write the checks

18

Lab 8: Gates that block a bad change

19

7. Week 5: Pipelines and MLOps

21

Lab 9: A nightly pipeline that re-indexes only what changed

21

Lab 10: Evaluation harness, drift signal and cost view

23

8. Week 6: Privacy and capstone

26

Lab 11: Entitlements, PII and an auditable answer

26

Lab 12: Capstone, one question through the whole system

27

9. Troubleshooting

29

Appendix A. Command cheat sheet

30

Appendix B. Running the labs on a Windows machine

31

B.1 Open the lab machine and a Command Prompt

31

B.2 Check that Python is installed

31

B.3 Get the lab pack

31

B.4 Run the setup script

31

B.5 Every time you open a new Command Prompt

32

B.6 Add the Azure track

32

(Right-click this table and choose Update Field if page numbers are missing.)

1. How to use this guide

Every lab in the programme runs on Azure Databricks. Section 2 sets that up, and it takes about five minutes. The same lab, the same data and the same code can also be run two other ways: on a Windows machine at a command prompt, and against live Azure services by adding `--azure` to a command. The table below compares those two.

	Command-line track	Azure track
What runs where	Everything runs on your lab machine	The same steps call Azure services
What you need	Python and the lab pack. No keys, no internet after setup	The lab pack plus a settings file called <code>lab.env</code> from your trainer
Models used	Small offline stand-ins, honest about their limits	Azure OpenAI (gpt-4.1-mini, text-embedding-3-small), Azure AI Search, Document Intelligence, AI Language, Application Insights
Cost	Free	Paid by the course subscription: about 0.44 US dollars per student for all 12 labs
Example	<code>python labs\lab03_hybrid.py</code>	<code>python labs\lab03_hybrid.py --azure</code>

Do the Databricks track first in every lab. It needs no setup and it cannot fail on a missing package. The command-line track is the same code without a platform, and the Azure track is the same lab against live Azure services. The difference between two runs is often the lesson.

1.1 Conventions

- On Databricks you run notebook cells, not commands. Where this guide shows a command, it belongs to the command-line track in appendix B.
- On Databricks there is nothing to activate. On a Windows machine, every command assumes the lab folder and an active environment, which appendix B.5 sets up.
- Grey boxes marked What you should see are real output from our test runs. Your timings will differ. Your numbers should match, because the data is generated the same way every time. All twelve labs were run end to end on Azure Databricks on 24 September 2026 and gave the same numbers.
- **Student id:** on the Azure track your work is tagged with your student id, so 25 people can share one search index without overwriting each other.

1.2 Which labs are in which week

Week	Day	Lab	Topic
1	1	Lab 1	Ingest five file formats into one indexed corpus
1	2	Lab 2	Benchmark three vector stores and pick one
2	3	Lab 3	Hybrid retrieval, reranking, nDCG lift
2	4	Lab 4	A RAG API with verifiable citations
3	5	Lab 5	NL2SQL on a star schema, with an accuracy number
3	6	Lab 6	Semantic layer, narration and charts
4	7	Lab 7	Trace a wrong number (lineage), then write the checks
4	8	Lab 8	Quality gates that block a bad change
5	9	Lab 9	A nightly pipeline that re-indexes only what changed
5	10	Lab 10	Evaluation harness, drift signal and cost view
6	11	Lab 11	PII, entitlements and an auditable answer
6	12	Lab 12	Capstone: one question through the whole system

Note: The lab briefs (goal, steps and definition of done) are in the `handouts` folder of the lab pack. This guide covers how to run each lab on a machine and what you should see.

1.3 Between-session assignments

Three assignments sit in the gaps between sessions. Each one is individual and takes two to three hours. Each is given at the end of one session and is due at the start of the next.

A1: make the corpus answer the escalation question. Given on Day 2, due on Day 3. Builds on Labs 1 and 2. The brief is in the lab pack at `handouts/a1_brief.md`.

A2: extend the semantic layer to a new question. Given on Day 6, due on Day 7. Builds on Labs 5 and 6. The brief is at `handouts/a2_brief.md`.

A3: the silent regression drill. Given on Day 10, due on Day 11. Builds on Labs 8, 9 and 10. The brief is at `handouts/a3_brief.md`.

Watch out: Do the work in notebook cells, not by editing files. The lab folder at `/tmp/freshcart` is rebuilt every session, so a change you make to a file on disk is gone the next day. Adding an entry to a dictionary, replacing a function and adding a gate all work from a cell.

Hand in your notebook, exported with outputs: File, then Export, then IPython notebook, with Include outputs ticked. That is the same proof you upload for a lab.

2. Set up

Time: about 5 minutes on your first day, then nothing. There is no software to install.

2.1 What you need

- A browser. Chrome and Edge both work.
- The workspace address. For this course it is `https://adb-7405618222155856.16.azuredatabricks.net`
- A sign-in for that workspace, which your trainer gives you.

Note: Every lab runs inside Azure Databricks. You do not install Python, you do not download the lab pack, and there is no setup script to run. Appendix B covers running the same labs on a Windows machine instead.

2.2 Sign in

1. Open the workspace address in your browser.
2. Click Continue with Microsoft Entra ID. Entra ID is the Microsoft sign-in service that Azure uses.
3. Sign in with the account your trainer gave you. Approve the second step on your phone if you are asked for one.

2.3 Make your own copy of the notebooks

1. In the left sidebar, click Workspace.
2. Open Workspace, then Shared, then freshcart-labs.
3. Click the three dots beside freshcart_notebooks and choose Clone.
4. Choose your own Home folder as the destination.
5. Open your copy. You should see twelve notebooks, lab01_ingest to lab12_capstone.

Watch out: Work in your own copy, never in the shared folder. Everyone in the class can open the shared one, so two people running it at the same time overwrite each other's results.

2.4 Connect a notebook to a machine

1. Open lab01_ingest from your copy.
2. Top right, click Connect, then choose Serverless.

Serverless means Databricks finds and starts the machine for you. You never size it, start it or stop it.

Note: The first connection of the day takes two to four minutes while Databricks finds a machine. Every connection after that takes a few seconds.

2.5 The first two cells of every lab

Every lab notebook starts with the same two cells. Run both before anything else.

The first cell installs the five file readers the labs need:

```
%pip install -q pypdf python-docx python-pptx openpyxl rank_bm25
```

It takes about 30 seconds. It then prints a note asking you to restart the kernel. Ignore that note. The labs import these five readers for the first time after the install, so a restart is not needed.

The second cell copies the lab pack out of shared storage into your own session, and puts the lab code where Python can find it.

What you should see:

```
lab folder: /tmp/freshcart
```

```
corpus files: 7
```

Watch out: If the second cell prints corpus files: 0, the lab pack did not unpack. Run the cell again. If it still says 0, tell your trainer.

2.6 How to run a lab

Run the cells from the top with Shift and Enter. Edit only the cells marked YOUR CALL. To run a whole notebook in one go, click Run all at the top.

Python on this compute is version 3.12.3. Every Databricks number in this guide was measured on it.

When you finish a lab, scroll to the **Your write-up** cell at the bottom, double click it, and type your answers there. Then export the notebook: File, then Export, then IPython notebook, with Include outputs ticked. That export is what you hand in. Do not write in the handouts folder, because /tmp/freshcart is rebuilt every session and anything you type there is gone tomorrow.

2.7 Where everything lives

```
notebooks    /Workspace/Shared/freshcart-labs/freshcart_notebooks
lab pack     /Volumes/aieng_dbx_itc/freshcart/labpack/labpack.zip
your copy    /tmp/freshcart
catalog      aieng_dbx_itc.freshcart
```

/tmp/freshcart belongs to your session alone. Nothing you write there is visible to anyone else, and it is cleared when the session ends. To start a lab from a clean copy, delete that folder and run the second cell again.

The second cell also checks the pack's date. If your trainer uploads a corrected pack, the next run of that cell unpacks it over the top and prints the lab pack on the volume changed, so this folder was refreshed from it. Files you made yourself stay where they are.

To look inside that folder, run this in any cell of the notebook that unpacked it:

```
%sh ls -R /tmp/freshcart
```

Note: It has to be the same notebook. Every notebook gets its own machine and its own /tmp, so a second notebook cannot see the first one's copy. dbutils.fs cannot read it either: dbutils.fs.ls on a file path answers Cannot access non /Workspace local filesystem. To open one of the documents on your own screen, copy it to the volume and download it from Catalog Explorer, for example shutil.copy("/tmp/freshcart/data/raw/refund_policy_v3_2024.pdf", "/Volumes/aieng_dbx_itc/freshcart/labpack/").

3. Week 1: Embeddings and vector stores

Lab 1: Ingest five formats into one indexed corpus

Goal. Load seven files in five formats, check what each loader really extracted, chunk them, index them, and run five test queries. Find the two silent failures: a scanned PDF that extracts nothing, and a superseded policy that outranks the current one.

Databricks track

Open `lab01_ingest` in your copy of `freshcart_notebooks` and click Run all.

Measured on Azure Databricks serverless on 24 September 2026: 49 seconds from the first cell to the last. Every number below matches the command-line run.

What you should see (the lines that carry the lesson):

```
lab folder: /tmp/freshcart
corpus files: 7
support_escalation_sop_scanned.pdf          1      0  EMPTY, needs OCR
total chunks: 30
indexed 30 chunks, dimension 29
    0.942  refund_policy_v2_2022.pdf      <-- SUPERSEDED
    0.795  refund_policy_v3_2024.pdf
```

The two findings are the same on both platforms. The scanned document extracts nothing and raises no error, and the 2022 policy beats the 2024 one until the filter is switched on.

Command-line track: the notebook

Lab 1 and Lab 2 run as notebooks, one step at a time. Each step has three cells: look at the real thing, write your own prediction or setting in the cell marked **YOUR CALL**, then reveal the measurement and the gap between the two.

1. In the lab folder, open the editor: `code .`

Open the `lab01_ingest` notebook in your workspace.

2. Top right, choose **Select Kernel**, then **Python Environments**, then the `.venv` one.
3. Run the cells from the top with Shift and Enter. Edit only the cells marked YOUR CALL.

Note: Prefer the plain script, or no editor? `python starter\lab1_starter.py` works through the same TODOs, and `python solutions\lab1_solution.py` is the reference. Past the 45 minute mark, read the reference rather than stall.

What you should see:

```
=== verification ===
partner_agreement_extract.pdf          units=1  chars=1190  ok
refund_claims_march_2024.xlsx          units=9  chars=1298  ok
refund_policy_v2_2022.pdf              units=1  chars=1174  ok
refund_policy_v3_2024.pdf              units=1  chars=1687  ok
support_escalation_sop_scanned.pdf     units=1  chars=0     EMPTY, needs OCR
support_qbr_q1_2024.pptx               units=3  chars=656   ok
support_sla_matrix.docx                units=1  chars=1247  ok

Q: how many refund claims can a customer make in thirty days
unfiltered:
    0.942 refund_policy_v2_2022.pdf      <-- SUPERSEDED
    0.795 refund_policy_v3_2024.pdf
current only:
    0.795 refund_policy_v3_2024.pdf
```


Two things to notice. The scanned SOP extracts **0 characters** and nothing reports an error. And the 2022 policy (superseded) scores 0.942 against 0.795 for the current one: similarity is working perfectly and still gives the wrong document. The recency filter fixes the second problem with one metadata field.

Step 7 of Lab 1: the same pieces through three embedders

Before the framework comparison, Lab 1 puts the same 30 pieces through three embedders and asks the same five questions of each. An embedder turns a piece of text into a list of numbers. `tfidf` and `hashing` are offline stand-ins. `bge` is BAAI/bge-small-en-v1.5, a current open-source retrieval model. It needs no key and no account: the first cell of the notebook installs it, and the model itself downloads once per session, about 130 MB.

What you should see:

embedder	dimensions	claims question	withdrawn document first
tfidf	29	0.942 vs 0.795	on 2 of the 5 questions
hashing	256	0.372 vs 0.319	on 3 of the 5 questions
bge-small-en-v1.5	384	0.833 vs 0.832	on 3 of the 5 questions

That last pair is the point of the step. One thousandth of a point separates the withdrawn 2022 policy from the one in force. A current retrieval model does not have a weak opinion about which is correct. It has none at all, and it cannot have one, because the two documents say nearly the same words. More dimensions is not better either: hashing has 256 and is the worst of the three.

The last cell then withdraws one file from the corpus and re-embeds a piece whose own text did not change. `tfidf` keeps 0.635 of the vector it had. `hashing` and `bge` keep 1.000. A vector that moves when other documents change cannot be cached, so a nightly load re-embeds everything. That is the bill Lab 9 pays.

Step 8 of Lab 1: the same seven files through LangChain and LlamaIndex

Lab 1 then runs two frameworks over the same seven documents and puts their answers beside the loaders in `src/freshcart.py`. The comparison code is `src/loaders_compare.py`. Both frameworks and the embedding model install in the notebook's first cell, which is the slowest cell in the lab.

What you should see:

source	ours		langchain		llamaindex	
	units	chars	units	chars	units	chars
partner_agreement_extract.pdf	1	1190	1	1190	1	1190
refund_claims_march_2024.xlsx	9	1298	–	–	1	2415
refund_policy_v2_2022.pdf	1	1174	1	1174	1	1174
refund_policy_v3_2024.pdf	1	1687	1	1687	1	1687
support_escalation_sop_scanned.pdf	1	0	1	0	1	0
support_qbr_q1_2024.pptx	3	656	–	–	3	800
support_sla_matrix.docx	1	1247	1	991	1	991

Four things to take from that table.

LangChain read five of the seven files. Its PowerPoint and Excel loaders both go through a package called `unstructured`, which is a much bigger install, so this step leaves them out and counts that as a result. LlamaIndex read all seven.

All three returned zero characters for the scanned PDF, and not one of them said so. The silent failure from step 1 is not something a framework saves you from. LlamaIndex does carry `extraction_errors` and `extraction_warnings` fields. It attaches them to the PowerPoint, which worked, and not to the scanned PDF, which did not.

The Word file gives 1,247 characters from your loader and 991 from both frameworks. The missing 256 characters are the column names, which your loader repeats beside every value. Ask yourself whether you could answer the question *what is the first response time for a P2* from the framework's version of that table.

The Excel file gives nine self-describing rows from your loader and one block from LlamaIndex, whose column names all read `Unnamed`. It assumed the first row was the header, and the first row is a title banner.

Note: The install is about 40 seconds and about 100 extra packages. That buys you a reader for every format. It does not buy you the other two jobs: keeping the structure that makes a piece answerable, and counting what actually came out.

Azure track

```
python labs\lab01_azure.py --azure
```

The same pipeline on Azure: Document Intelligence reads the scanned page, Azure OpenAI embeds every chunk into 1,536 numbers, and the chunks go into a shared Azure AI Search index tagged with your student id.

What you should see:

```
=== 2. OCR the source that extracted nothing (Document Intelligence, prebuilt-read) ===
support_escalation_sop_scanned.pdf      1      804  ok
OCR took 4.8s (cached after the first run). First 300 characters:
FreshCart Support Escalation SOP Document SOP-SUP-014 : Revision 5 : March 2024 1. When to
escalate to a supervisor A support agent must escalate to a supervisor where the
customer ...

=== 3. chunk and embed with Azure OpenAI ===
31 chunks -> vectors of 1536 numbers each
tokens sent: 2241 (cache hits: 0), about $0.000045

=== 4. index into Azure AI Search ===
index 'freshcart-chunks': removed 0 old documents, uploaded 31 for student 'trainer'

Q: how many refund claims can a customer make in thirty days
unfiltered:
  0.781  refund_policy_v2_2022.pdf      3. Claim frequency limits ...  <-- SUPERSEDED
  0.773  refund_policy_v3_2024.pdf      3. Claim frequency limits ...
Q: when must an agent escalate to a supervisor
unfiltered:
  0.715  support_escalation_sop_scanned.pdf  FreshCart Support Escalation SOP ...
```

Compare with the command-line run: the SOP is no longer empty, so the escalation question now finds the document that actually answers it. OCR text is cached in `data\cache`, so the second run does not pay for OCR again.

The superseded 2022 policy still scores above the current one with real embeddings (0.781 against 0.773). Better embeddings do not fix a metadata problem; the recency filter does.

Lab 2: Benchmark three vector stores and pick one

Goal. Measure recall@10, p50 and p95 latency, build time and memory for three stores; sweep the query-time dial; test what a selective filter does to result counts; write a one-page decision memo.

Databricks track

Open `lab02_vector_stores` in your copy of `freshcart_notebooks` and click Run all.

Measured on Azure Databricks on 24 September 2026: 27 seconds.

What you should see:

```
{'store': 'flat-exact', 'recall@10': 1.0, 'p50_ms': 0.01, 'p95_ms': 0.013, 'memory_kb': 3.4}
{'store': 'ivf', 'recall@10': 0.49, 'p50_ms': 0.028, 'p95_ms': 0.082, 'memory_kb': 4.3}
{'store': 'sqlite', 'recall@10': 1.0, 'p50_ms': 0.05, 'p95_ms': 0.231, 'memory_kb': 3.4}

n_probe  recall@10  p50_ms  p95_ms
```

1	0.280	0.018	0.024
2	0.490	0.021	0.024
3	0.720	0.014	0.018
4	0.840	0.017	0.026
6	0.940	0.024	0.037
8	1.000	0.022	0.024

Note: The notebook sweeps six settings and the script in the command-line track sweeps four, so the two tables are not line for line the same. The shape is: recall climbs from 0.28 to 1.00 while the milliseconds barely move.

The chart is written to `/tmp/freshcart/out/lab2_curve.png` and also drawn in the notebook.

Command-line track: the notebook

Open the `lab02_vector_stores` notebook in your workspace, the same way as Lab 1. You commit to a prediction before each measurement: which store will be fastest, what recall the clustered index gives when it probes two groups of eight, and where the curve stops paying. The notebook saves the recall against latency chart to `out/lab2_curve.png` for your memo.

```
python starter\lab2_starter.py
python solutions\lab2_solution.py
```

What you should see:

store	recall@10	p50_ms	p95_ms	build_s	memory_kb
flat-exact	1.0	0.003	0.004	0.0	3.4
ivf	0.492	0.008	0.01	0.021	4.3
sqlite	1.0	0.019	0.02	0.0	3.4


```
n_probe sweep (the recall/latency curve):
n_probe  recall@10  p50_ms  p95_ms
1         0.292    0.007  0.009
2         0.492    0.008  0.008
4         0.817    0.008  0.011
8         1.0     0.009  0.01
```

The curve is the finding: recall climbs from 0.29 to 1.0 as `n_probe` rises, latency barely moves at this size, and memory does not change at all because `n_probe` is a query-time setting. In the starter, the post-filter test returns 6, 8 and 9 results when 10 were asked for; the pre-filter always returns 10.

Step 7 of Lab 2: the dial on somebody else's service

Lab 2 ends by asking what the curve you just swept is worth on a service that does not let you turn that dial. The code is `src/managed_store.py`.

What you should see:

setting	recall@10	answers you lose in 10
n_probe 8 (you chose it)	1.000	0.0
n_probe 1 (a vendor default)	0.280	7.2
n_probe 2 (a vendor default)	0.490	5.1
n_probe 4 (a vendor default)	0.800	2.0

On this corpus, a vendor default of 1 costs you 7.2 right answers out of every 10, and nothing on screen tells you. The step then prints which controls Databricks AI Search, Azure AI Search and Fabric or Synapse actually expose, and the three jobs that stay yours whichever one you buy.

Note: The first and last rows are the same everywhere. The middle two can move by about 0.02 recall between package versions, because the grouping step is seeded per build. The shape of the answer does not change.

Azure track

```
python labs\lab02_azure.py --azure
```

Exact search on your machine is the ground truth. Azure AI Search is measured with its HNSW index and with exhaustive search, over the network. Add `--pgvector` if your trainer has set up PostgreSQL (trainer guide, section 10.5).

What you should see:

```
=== pre-filter vs post-filter in Azure AI Search (keep only SUPERSEDED chunks) ===
preFilter mean results returned: 7.0 (asked for 10; 7 chunks match the filter)
postFilter mean results returned: 1.8 (asked for 10; 7 chunks match the filter)

=== results ===
store          vectors recall@10 p50_ms p95_ms
flat (local, exact) 31      1.000    0.100  0.100
AI Search HNSW    31      1.000   212.500 231.900
AI Search exhaustive 31      1.000   208.600 225.600
```

Two findings. With the selective filter, post-filtering returned 1.8 results on average when 10 were asked for, while pre-filtering returned all 7 chunks that match. And HNSW and exhaustive search both take about 210 ms, against 0.1 ms locally: on a corpus this small the network decides latency, not the algorithm.

Write the memo in the **Your write-up** cell at the bottom of the Lab 2 notebook, and say what would change your decision at 10 million vectors.

4. Week 2: Retrieval and grounding

Lab 3: Hybrid retrieval, reranking and the lift

Goal. Measure nDCG@10 at four stages, one at a time: dense vectors only, hybrid (dense plus BM25 keyword search fused by reciprocal rank), plus reranking, plus a recency filter. Relevance judgements for 20 queries are in `data\golden\retrieval_golden.json`.

Databricks track

Open `lab03_hybrid` in your copy of `freshcart_notebooks` and click Run all.

Measured on Azure Databricks on 24 September 2026: 28 seconds. The numbers match the command-line run below.

Command-line track

```
python labs\lab03_hybrid.py
```

What you should see:

```
=== summary: the lift, one stage at a time ===
stage  mean_nDCG@10  mean_nDCG@3  p50_ms  p95_ms
dense   0.803         0.785         0.200    0.200
hybrid  0.806         0.788         0.300    0.300
rerank  0.815         0.794         0.500    0.500
filter  0.874         0.855         0.700    0.700

=== queries that got WORSE after reranking (explain one in your debrief) ===
R02 who pays when food is spilled in transit    hybrid 0.31 -> rerank 0.26
R05 what is the first response target for a spil hybrid 0.71 -> rerank 0.59
```

On this 30-chunk corpus each retriever returns almost everything, so hybrid adds little at @10; the recency filter adds the most. The offline reranker is a lexical stand-in and is roughly neutral: report that, do not hide it. Try `K_EACH = 10` at the top of the script and watch hybrid start to matter.

Azure track

```
python labs\lab03_hybrid.py --azure
```

Dense and hybrid queries run inside Azure AI Search (the service fuses keyword and vector results with reciprocal rank fusion). The reranker is gpt-4.1-mini grading each passage from 0 to 3.

What you should see:

```
=== summary: the lift, one stage at a time ===
stage  mean_nDCG@10  mean_nDCG@3  p50_ms  p95_ms
dense   0.832         0.798         247.600  296.000
hybrid  0.886         0.879         240.000  301.000
rerank  0.924         0.916         3876.400 7234.500
filter  0.987         0.981         3796.900 4724.400

=== Week 1 failures ===
R01 how many refund claims can a customer make i dense 0.64 -> final 0.98 improved
R02 who pays when food is spilled in transit    dense 0.27 -> final 0.98 improved
R10 CLM-2024-0314                               dense 0.50 -> final 1.00 improved

Azure OpenAI usage: 40 chat calls, 48830 tokens in, 7360 out, about $0.0313
```

With real embeddings and a model reranker, every stage adds lift, from 0.832 to 0.987, and the three Week 1 failures are fixed. The price is latency: reranking takes about 3.9 seconds a query at the median, against 0.24 seconds for hybrid search. Decide in your debrief whether that is worth it for a support agent waiting on a live call.

Lab 4: A RAG API with verifiable citations

Goal. Serve answers through an HTTP endpoint; cite chunk ids; check in code that every cited id was actually retrieved; refuse below a relevance threshold; report refusal rates for answerable and unanswerable questions separately.

Databricks track

Open `lab04_rag_api` in your copy of `freshcart_notebooks` and click Run all.

Measured on Azure Databricks on 24 September 2026: 56 seconds. The numbers match the command-line run below.

Watch out: The step called Run it as an API does not work on Databricks. A notebook cannot open a web address that your browser can reach. Do that one step on a Windows machine using appendix B, or watch your trainer do it.

Command-line track

```
python labs\lab04_rag_api.py
python labs\lab04_rag_api.py --threshold 0.6
python labs\lab04_rag_api.py --inject-phantom
```

What you should see:

```
=== the numbers for your debrief ===          (default threshold 0.15)
refusal rate, answerable      : 0/20  (should be LOW)
refusal rate, unanswerable    : 0/5   (should be HIGH)
faithfulness, answered       : 1.000
correct facts, answerable    : 17/20
phantom citations caught     : 0
```

The default threshold refuses none of the five unanswerable questions. That is deliberate: tune `--threshold` until unanswerable questions are refused without refusing answerable ones. `--inject-phantom` makes every third answer cite a chunk that was never retrieved; the check catches all of them (8 in our run).

Run it as an API

1. Start the server. Leave this window open: `python labs\lab04_rag_api.py serve`
2. Open a second Command Prompt, run the two lines from appendix B.5, then ask a question: `python labs\lab04_rag_api.py ask "How many refund claims can a customer make?"`
3. Or open `http://127.0.0.1:8000/docs` in the lab machine's browser, choose **POST /ask**, then **Try it out**.
4. Stop the server with Ctrl+C in its window.

Azure track

```
python labs\lab04_rag_api.py --azure
python labs\lab04_rag_api.py serve --azure
```

What you should see:

```
=== the numbers for your debrief ===
refusal rate, answerable      : 0/20  (should be LOW)
refusal rate, unanswerable    : 4/5   (should be HIGH)
faithfulness, answered       : 1.000
correct facts, answerable    : 20/20
phantom citations caught     : 0

(with --inject-phantom: phantom citations caught : 7)
Azure OpenAI usage: 47 chat calls, 38989 tokens in, 4727 out, about $0.0232
```

At the same default threshold, 4 of the 5 unanswerable questions are refused, and all 20 answerable ones get the right facts. Three refusals come from the threshold, because the model reranker grades every passage as unrelated; one comes from the answering model saying it could not find the answer. The command-line stand-in refused none, so measure both refusal rates again after every model change.

5. Week 3: NL2SQL and semantic layers

Lab 5: NL2SQL on the star schema, with an accuracy number

Goal. Turn 16 business questions into SQL, run each one through safety rails (a single read-only SELECT, row limit, timeout), and score by comparing result sets. Measure three conditions: schema only; plus business definitions; plus four few-shot examples. Classify every failure.

The warehouse has five planted traps: a deprecated `orders` table, a meaningless `status` column on customers, internal test accounts (customers 1 and 2) that must be excluded, a one-to-many `order_items` table that inflates totals, and a cumulative month-end snapshot that double counts if summed.

Databricks track

Open `lab05_nl2sql` in your copy of `freshcart_notebooks` and click Run all.

Run end to end on Azure Databricks on 24 September 2026. The warehouse file travels inside the lab pack, so the three accuracy runs give the same 7, 10 and 14 out of 16 as the command-line track.

Command-line track

```
python labs\lab05_nl2sql.py
```

A laptop cannot run a large language model, so the command-line track replays SQL that gpt-4.1-mini actually wrote for these questions on 15 September 2026. Execution, scoring and classification run live on your machine.

What you should see:

```
=== accuracy by condition ===
condition          accuracy  valid_but_wrong  invalid
A  schema only      7/16         8                1
B  + definitions    10/16        5                1
C  + definitions + few-shot 14/16        1                1

failure classes
A  missing filter 7, schema retrieval 1, wrong definition 1
B  ambiguous time 1, missing filter 4, schema retrieval 1
C  ambiguous time 1, schema retrieval 1
```

Business definitions lift accuracy from 44% to 63%; worked examples lift it to 88%. Two failures remain in every condition: the schema retriever never offers `dim_restaurant` to the model (a retrieval failure, not a model failure), and "last month" has no agreed meaning (definitions cannot fix ambiguity).

Azure track

```
python labs\lab05_nl2sql.py --azure
```

The live model writes the SQL. Expect small differences from the recorded run; that run-to-run spread is the noise floor you will measure in Lab 8. The live run costs about 1.5 US cents.

Lab 6: Natural language to insight, with a semantic layer

Goal. Declare four metrics once (joins, filters, grain), route questions to a metric instead of free SQL, narrate the number with a comparison and its scope, and draw a chart with a zero baseline.

Databricks track

Open `lab06_semantic_layer` in your copy of `freshcart_notebooks` and click Run all.

Run end to end on Azure Databricks on 24 September 2026. The lab code is the same file on both platforms, so the numbers in the command-line block below are what you should see.

Command-line track

```
python labs\lab06_semantic_layer.py
```

What you should see:

```
=== denominator check on refund_rate (claims per order vs orders with a claim) ===
March 2024: claims / orders = 0.0452    orders with a claim / orders = 0.0409

=== Lab 5 questions through the layer ===
answered by the layer: 4/16, correct: 4/4

Q: How many refund claims were there in March 2024?
Refund Claims was 21 in 2024-03. That is down from 28 in 2024-02. Scope: excluding
internal test accounts.
```

Charts are saved in the `out` folder (`lab06_chart_1.png` and so on); open them from File Explorer. The layer answers 4 of the 16 Lab 5 questions and gets all 4 right. It cannot answer the other 12 at all: that is the honest cost of the approach.

Step 7 of Lab 6: the same metric, declared in dbt

A small dbt project over the same warehouse, one model per metric, run with DuckDB so it needs no server and no network. The code is `src/dbt_compare.py`.

Watch out: dbt is not installed on Databricks. Its install takes well over ten minutes inside a notebook, so the step prints the install command and the recorded numbers instead of running. On a Windows machine the setup script installs it, and the step runs live in about four seconds. A trainer who wants it live on Databricks should add `dbt-core` and `dbt-duckdb` to the workspace base environment once, under Settings, then Compute.

What you should see:

```
Done. PASS=11 WARN=0 ERROR=0 SKIP=0 TOTAL=11
```

metric	lab 6 layer	dbt model	same
revenue	6292927.260	6292927.260	True
order_count	5546.000	5546.000	True
refund_claims	277.000	277.000	True
refund_rate	0.050	0.050	True

Two declarations, four identical numbers. dbt adds tests, lineage and documentation that you did not write. It has no way to refuse a question no metric answers, which is the thing your layer did in step 3. The step ends with the table that says which does what.

Azure track

```
python labs\lab06_semantic_layer.py --azure
```

gpt-4.1-mini maps each question to a metric, dimension and period in JSON. It never writes SQL. The script then checks the plan: the question must name the metric it was sent to. Without that check, the model sent N06 (lifetime value in a snapshot) to the revenue metric in our test, and the layer returned a confident, wrong number.

What you should see:

```

=== Lab 5 questions through the layer ===
id   metric      dim      period              correct
---   -
N01  order_count    2024-05-01..2024-05-31  True
N04  revenue        2024-01-01..2024-01-31  True
N09  revenue        city    2024-04-01..2024-06-30  True
N15  revenue        region 2024-07-01..2024-09-30  True

answered by the layer: 4/16, correct: 4/4
the layer CANNOT answer (this is the honest cost of the approach):
  N02  How many distinct customers placed an order in April 2024?
  ...
  N06  What was the total lifetime value of all customers in the June 2024 snapshot?
        guard: routed to revenue, but the question never names that metric
  ...

Q: Show the monthly order count for 2024.
  plan: {'metric': 'order_count', 'dimension': 'month', 'start': '2024-01-01', 'end':
'2024-12-31', ...}
  2024-01: 417
  2024-02: 447
  ...
  2024-12: 481
  chart (line): C:\Users\admin\freshcart-labs\out\lab06_chart_2.png

Q: How many refund claims were there in March 2024?
  Refund Claims was 21 in 2024-03. That is down from 28 in 2024-02. Scope: excluding
  internal test accounts.

Azure OpenAI usage: 20 chat calls, 5480 tokens in, 835 out, about $0.0035

```

6. Week 4: Data quality and governance

Lab 7: Trace a wrong number, then write the checks

Goal. Two reports disagree about customer 57 in March 2024. Trace both numbers to their source columns, decide which is right, find a defect nobody described, then write expectations with an action for each (fail, quarantine or warn) and an owner per dataset.

Databricks track

Open `lab07_lineage` in your copy of `freshcart_notebooks` and click Run all.

Run end to end on Azure Databricks on 24 September 2026. The lab code is the same file on both platforms, so the numbers in the command-line block below are what you should see.

Command-line track

```
python labs\lab07_lineage.py
```

What you should see:

```
=== 1. the disagreement: customer 57, March 2024 ===
report_claims_by_customer    -> 4
report_orders_with_claims    -> 3

=== 3. where they diverge: orders claimed more than once ===
27 orders have more than one claim row. First five:
  order 6987 customer 57: 2 rows [2024-03-21 Declined | 2024-03-22 Under review]
whole register: 277 claim rows, 250 distinct orders

=== 4. profile the data: one more defect is not described anywhere ===
claims whose order is cancelled      23

=== 5. expectations ===
unique_open_claim_per_order    False  quarantine  27 orders with more than one claim row
no_claims_on_cancelled_orders  False  warn         23 claims on cancelled orders
volume(orders_v2)              False  fail         9 vs mean 18.9
```

One report counts claim rows; the other counts orders with a claim. A resubmitted claim is one order and two rows. The volume check fails because 31 December had 9 orders against a 7-day mean of 18.9: decide whether that is a partial load or ordinary variation, and set the tolerance accordingly.

Azure track: Azure Databricks Unity Catalog

In Unity Catalog, lineage is captured automatically when tables are built. It runs in the course Databricks workspace from section 2.

1. In the workspace, open **Catalog**, pick the catalog your trainer names, and create a volume called `raw` in a new schema `freshcart_<your student id>`.
2. Upload `data\freshcart.db` from your lab folder into that volume.
3. Import `azure\databricks\lab07_unity_catalog_lineage.py` (**Workspace**, **Import**, choose the file). It opens as a notebook.
4. Set the two widgets at the top (catalog, student id), attach **Serverless** compute or a single-node cluster, and choose **Run all**.
5. Open `report_orders_with_claims` in Catalog Explorer, then **Lineage**, then **See lineage graph**. Click the column `orders_with_claim`: its source is `fact_refund_claim.order_id`. The other report's `claims` column comes from `fact_refund_claim.claim_id`.
6. The notebook adds a CHECK constraint and then tries to insert a refund of -250. That last cell is expected to fail: the table refuses the bad row.

Note: This notebook is included but has not yet been run in the course workspace. Treat its output as untested.

Lab 8: Gates that block a bad change

Goal. A CI script that runs the Lab 7 expectations, a schema contract, and an evaluation gate on the golden set, with thresholds set outside the measured noise. Three provided changes: two must be blocked, one must pass.

Databricks track

Open `lab08_gates` in your copy of `freshcart_notebooks` and click Run all.

Run end to end on Azure Databricks on 24 September 2026. The lab code is the same file on both platforms, so the numbers in the command-line block below are what you should see.

Command-line track

```
python labs\lab08_gates.py --update-baseline
python labs\lab08_gates.py --change schema
echo %ERRORLEVEL%
python labs\lab08_gates.py --change prompt
python labs\lab08_gates.py --change chunking
```

What you should see:

Change	What it does	Result	Exit code
none	Unchanged pipeline	merge ALLOWED	0
schema	An upstream team renames <code>amount</code> to <code>claim_amount</code>	merge BLOCKED: expectations cannot run, schema contract fails	1
prompt	A cautious edit: answer only when two passages support the answer	merge BLOCKED: refusal on answerable questions rose from 0.0 to 0.85. A faithfulness-only gate would have said PASS	1
chunking	Chunk overlap 120 to 100 characters	merge ALLOWED: nDCG 0.8739 to 0.8726, inside the noise	0

```
=== noise floor (spread on unchanged code) and the thresholds it implies ===
metric          noise_sd  tolerance
ndcg10          0.070    0.140
faithfulness    0.000    0.030
refusal_answerable 0.000    0.050
```

With only 20 golden queries the nDCG noise is large (0.07), so a tight nDCG gate would fail at random. The fix is more golden queries, not a looser gate. The reviewer-facing report is written to `out\gate_report.md`.

Why the prompt change refuses so much: most facts in the corpus appear in exactly one document, so a two-source rule refuses 17 of the 20 answerable questions. Every answer it still gives is faithful, which is why a faithfulness-only gate lets it through.

Note: The two-source rule is added to the prompt and also enforced in code, in `apply_change` in `labs\lab08_gates.py`. A prompt alone did not change behaviour. The offline stand-in ignores prompts, and on Azure gpt-4.1-mini still answered all 20 answerable questions under nine different cautious wordings we tested. Treat a prompt edit as a guess about behaviour until the evaluation has measured it.

Wire it into CI

`ci\github-actions.yml` and `ci\azure-pipelines.yml` run the same script on every pull request. A non-zero exit code fails the check and blocks the merge. Copy the GitHub file to `.github\workflows\quality-gates.yml` in a repository, or add the Azure Pipelines file as a pipeline and require it in the branch policy.

Azure track

```
python labs\lab08_gates.py --update-baseline --azure
python labs\lab08_gates.py --change schema --azure
python labs\lab08_gates.py --change prompt --azure
python labs\lab08_gates.py --change chunking --azure
```

What you should see:

```
=== change under test: prompt ===

=== noise floor (spread on unchanged code) and the thresholds it implies ===
metric          noise_sd  tolerance
ndcg10          0.014    0.030
faithfulness    0.000    0.030
refusal_answerable 0.000    0.050
refusal_unanswerable 0.140    0.280
citation_validity 0.000    0.000

=== gate report (what the reviewer sees) ===
gate              result  detail
...
schema contract: fact_refund_claim PASS    missing=[] retyped=[] added=[]
gate:ndcg10       PASS    current=0.9754 baseline=0.9754
gate:faithfulness PASS    current=1.0 baseline=1.0
gate:refusal_answerable FAIL    current=0.95 baseline=0.0
gate:refusal_unanswerable PASS    current=1.0 baseline=0.8
gate:citation_validity PASS    current=1.0 baseline=1.0

    a naive faithfulness-only gate would say: PASS
    merge BLOCKED: 1 blocking gate(s) failed

Azure OpenAI usage: 46 chat calls, 46445 tokens in, 6741 out, about $0.0294
```

Change	Result on Azure	Exit code
none	merge ALLOWED: nDCG 0.977 against a baseline of 0.9754	0
schema	merge BLOCKED: expectations cannot run, schema contract fails	1
prompt	merge BLOCKED: refusal on answerable questions rose from 0.0 to 0.95. A faithfulness-only gate would have said PASS	1
chunking	merge ALLOWED: nDCG 0.9754 to 0.9676, inside the tolerance of 0.03	0

Two differences from the command line. The nDCG noise is smaller (0.014 against 0.070), because nearly every query scores close to 1.0, so resampling the golden set moves the mean less. And the unanswerable refusal gate needs a tolerance of 0.28: with only 5 unanswerable questions, one changed answer moves the rate by 0.2. More unanswerable golden questions would let that gate be tighter.

7. Week 5: Pipelines and MLOps

Lab 9: A nightly pipeline that re-indexes only what changed

Goal. A pipeline with steps detect, ingest, verify, embed, index, evaluate, enrich and swap. It must prove four things: an unchanged run costs nothing, a changed document is the only one re-embedded, a deleted document leaves the index, and a crashed run resumes.

Databricks track

Open `lab09_pipeline` in your copy of `freshcart_notebooks` and click Run all.

Run end to end on Azure Databricks on 24 September 2026. The lab code is the same file on both platforms, so the numbers in the command-line block below are what you should see.

Command-line track

```
python labs\lab09_pipeline.py reset
python labs\lab09_pipeline.py run
python labs\lab09_pipeline.py run
python labs\lab09_pipeline.py simulate modify
python labs\lab09_pipeline.py run
python labs\lab09_pipeline.py simulate delete
python labs\lab09_pipeline.py run
python labs\lab09_pipeline.py simulate restore
python labs\lab09_pipeline.py run --kill-after embed
python labs\lab09_pipeline.py run
python labs\lab09_pipeline.py status
```

What you should see:

run_id	resumed	added	modified	deleted	embedded	reused	ndcg10	swapped
run001	False	7	0	0	30	0	0.663	True
run002	False	0	0	0	0	30	0.663	True
run003	False	0	1	0	2	28	0.676	True
run004	False	0	0	1	0	27	0.584	True
run005	True	1	1	0	0	30	0.663	True

Read the rows as the four proofs. Run 2 embedded nothing. Run 3 re-embedded 2 chunks from the one modified document. Run 4 removed the deleted deck (27 chunks, and the log prints `chunks ... in the live index = 0`). Run 5 crashed after embedding, then resumed and paid for nothing twice. The scanned SOP is quarantined on every run because it needs OCR.

Schedule it every night

```
python labs\lab09_pipeline.py schedule
```

This prints a `schtasks` command with your own paths. Paste it to create a Windows Task Scheduler job that runs at 02:00; the output also shows how to run it now and how to delete it.

Step 7 of Lab 9: an orchestrator around the pipeline you wrote

The same seven pipeline commands as a Prefect flow, including the one that crashes on purpose. The code is `src/orchestrate.py`, and it also writes the equivalent Airflow DAG and Databricks job so you can read all three.

What you should see:

task	exit_code
reset	0
first_run	0
modify_source	0
incremental	0
crash	1

```
resume      0
status      0
```

The flow takes about a minute. Read the comparison table's second row: resume is the one thing none of the three orchestrators does for you. The checkpoint you wrote in step 4 is still the reason it works.

Step 8 of Lab 9: the size at which one machine stops being enough

200,000 support tickets, generated rather than shipped, and one question asked of them two ways. The code is `src/tickets.py` and `src/scale_compare.py`.

What you should see:

```
200,000 tickets in 1.6s, 21.7 MB

one machine: 1.96s, peak 7.8 MB

priority tickets breaches breach_rate p95_resolution_minutes
P1      12104      2215      0.183      2273
P2      36173      6440      0.178      2339
P3      92199     16751      0.182      2607
P4      59524     10643      0.179      7655
```

Spark answers the same question on Databricks and takes longer, because at 200,000 rows you are paying for a cluster you do not need. That is the honest finding, and it is the one to take into a design review. The reason to move is the next zero, not today's seconds.

```
how Spark got the data: the engine could not open the notebook machine's disk,
so the rows were sent to it. The seconds below include that transfer.

spark: 6.63s against one machine 4.61s

counts identical: True
  P3: p95 differs by 1 minutes (2607 exact against 2606 approximate)
  P4: p95 differs by 3 minutes (7655 exact against 7652 approximate)
```

Note: On Databricks serverless compute, Spark runs on machines other than your notebook, so it cannot open a file in `/tmp` on the notebook machine. The step hands it the three columns the question needs instead, and says so in its first line. That transfer is part of the 6.63 seconds. Every count matches the one-machine answer exactly. Only the 95th percentile differs, by 1 to 3 minutes, because Spark's `percentile_approx` is approximate by design.

Note: The Spark cell is skipped on a Windows machine, which has no Spark. Everything else in this step runs on both. On Databricks the two lines above read 200,000 tickets in 3.1s and one machine: 4.61s, against 1.6s and 1.96s on the Windows lab machine. Compare the two routes inside one machine, not the two machines with each other.

Azure track

Run the same sequence as the command-line track, with `--azure` added to every line. `reset` also works with `--azure`, and the first run re-uses nothing:

```
python labs\lab09_pipeline.py reset --azure
python labs\lab09_pipeline.py run --azure
python labs\lab09_pipeline.py run --azure
python labs\lab09_pipeline.py simulate modify --azure
python labs\lab09_pipeline.py run --azure
python labs\lab09_pipeline.py simulate delete --azure
python labs\lab09_pipeline.py run --azure
python labs\lab09_pipeline.py simulate restore --azure
python labs\lab09_pipeline.py run --kill-after embed --azure
python labs\lab09_pipeline.py run --azure
python labs\lab09_pipeline.py status --azure
```

On Azure, embeddings and claim enrichment use Azure OpenAI, the scanned SOP is read with OCR instead of being quarantined, and each swap is copied into Azure AI Search: new chunks, edited chunks and deletions.

What you should see:

```
python labs\lab09_pipeline.py status --azure
run_id resumed added modified deleted embedded reused embed_cost_usd ndcg10 swapped
run001 False 7 0 0 31 0 0.000 0.832 True
run002 False 0 0 0 0 31 0.000 0.832 True
run003 False 0 1 0 2 29 0.000 0.813 True
run004 False 0 0 1 0 28 0.000 0.722 True
run005 True 1 1 0 0 31 0.000 0.832 True

=== run001: detect ===
  verify: OCR applied to ['support_escalation_sop_scanned.pdf']
=== run001: embed ===
  31 chunks: 31 embedded, 0 reused | about 2232 tokens, $0.000045
=== run003: embed ===
  31 chunks: 2 embedded, 29 reused | about 358 tokens, $0.000007
=== run003: enrich ===
  Azure AI Search: 0 added, 2 updated, 0 deleted (owner trainer, index freshcart-chunks)
=== run004: enrich ===
  Azure AI Search: 0 added, 0 updated, 3 deleted (owner trainer, index freshcart-chunks)
=== run004: swap ===
  deletion proof: chunks from ['support_qbr_q1_2024.pptx'] in the live index = 0
=== run005: enrich ===
  Azure AI Search: 3 added, 2 updated, 0 deleted (owner trainer, index freshcart-chunks)
```

Read run 3 closely. The edited SLA matrix produced 2 new embeddings, but its chunk ids did not change, because an id comes from the first words of a chunk. A mirror that compared ids only would have left the old text in the search index, and nothing would report it. The script compares text as well, so it reports 2 updated. Run 4 removes the 3 chunks of the deleted deck from Azure AI Search too.

Lab 10: Evaluation harness, drift signal and cost view

Goal. Log every answer with five field groups (request, retrieval, generation, answer, cost), then inject three faults and show that each is caught by a different signal.

Databricks track

Open `lab10_observe` in your copy of `freshcart_notebooks` and click Run all.

Run end to end on Azure Databricks on 24 September 2026. The lab code is the same file on both platforms, so the numbers in the command-line block below are what you should see.

Note: The notebook does not use MLflow. The `--mlflow` option belongs to the command-line script and writes to a local file.

Command-line track

```
python labs\lab10_observe.py
python labs\lab10_observe.py --fault index
python labs\lab10_observe.py --fault prompt
python labs\lab10_observe.py --fault cost
```

What you should see:

Fault	What it simulates	Signal that fired	Evidence
index	The nightly load silently skipped two documents	retrieval score drift	mean top score 0.826 to 0.635
prompt	Grounding loosened: answers gain unsupported claims	faithfulness drop	1.0 to 0.813
cost	20 chunks of context per answer instead of 5	cost per query up more than 20%	\$0.000588 to \$0.001719

Per-answer logs are in `data\logs`. Add `--mlflow` to log each run to MLflow after `pip install -r requirements-mlops.txt`, then open the dashboard with `mlflow ui --backend-store-uri sqlite:///mlflow.db`.

Step 7 of Lab 10: every run, not just the last one

All four harness runs logged to MLflow and read back as one table. The code is `src/observe_tools.py`.

What you should see:

when	fault	mean_top_score	faithfulness	refusal_rate	cost_per_query_usd
23:51:16	prompt	0.751	0.813	0.000	0.001
23:51:16	none	0.751	1.000	0.000	0.001
23:51:16	index	0.591	1.000	0.000	0.001
23:51:16	cost	0.751	1.000	0.000	0.002

Each fault moves a different column, and all four sit in one view. Run the lab again and there will be eight rows rather than four. A single baseline file would still hold exactly one previous run.

Note: On Databricks the runs go to this notebook's own MLflow experiment, and the step says so in its first line: logging 4 runs to this notebook's MLflow experiment. Databricks also shows a (4) MLflow runs panel above the table, with a link to that experiment. On Windows there is no such experiment, so the runs go to one SQLite file at `out/mlflow.db`, which needs no server. The `when` column is the clock of the machine that ran it, which is UTC on Databricks.

Step 8 of Lab 10: a drift library on the same numbers

Evidently is given the same retrieval scores your first signal compares, so the two can be read side by side.

What you should see:

index	ours	fired=True	(0.7514 -> 0.5906)	evidently drift=True	test=K-S p=0.000591
prompt	ours	fired=False	(0.7514 -> 0.7514)	evidently drift=False	test=K-S p=1.0
cost	ours	fired=False	(0.7514 -> 0.7514)	evidently drift=False	test=K-S p=1.0

Evidently replaces a threshold you guessed with a test that has a name, and it picked that test for you. It does not replace the decision about which column to watch. Both miss the prompt fault and the cost fault for the same reason: neither lives in the retrieval scores.

Azure track

```
python labs\lab10_observe.py --azure
python labs\lab10_observe.py --fault index --azure
python labs\lab10_observe.py --fault prompt --azure
python labs\lab10_observe.py --fault cost --azure
```

What you should see:

```
python labs\lab10_observe.py --fault index --azure
FAULT index: 9 of 31 chunks missing (partner_agreement_extract.pdf,
refund_policy_v3_2024.pdf)
signal      fired  evidence
retrieval score drift    True  mean top score 0.84 -> 0.56
faithfulness drop > 0.05  True  0.9972 -> 0.8636
refusal rate moved > 0.10 True  answerable 0.0 -> 0.45, unanswerable 0.8 -> 1.0
cost per query up > 20%   False $0.001184 -> $0.001160

python labs\lab10_observe.py --fault prompt --azure
signal      fired  evidence
retrieval score drift    False mean top score 0.84 -> 0.8533
faithfulness drop > 0.05  True  0.9972 -> 0.8153
refusal rate moved > 0.10 True  answerable 0.0 -> 0.0, unanswerable 0.8 -> 0.0
cost per query up > 20%   False $0.001184 -> $0.001236

python labs\lab10_observe.py --fault cost --azure
signal      fired  evidence
retrieval score drift    False mean top score 0.84 -> 0.84
faithfulness drop > 0.05  False 0.9972 -> 0.9762
refusal rate moved > 0.10 False answerable 0.0 -> 0.0, unanswerable 0.8 -> 0.8
cost per query up > 20%   True  $0.001184 -> $0.001727
```

Each fault still has its own first signal: drift for the index, faithfulness for the prompt, cost for the context size. With a real model the index fault fires three signals, not one. Without the current policy and the partner agreement in the index, the model refused 9 of the 20 answerable questions in this run (8 in a second run), and the answers it does give are less faithful.

Expect signals near a threshold to change between runs. In our second run the cost fault also dropped faithfulness from 0.994 to 0.937, just past the 0.05 line, because with 20 passages the model adds material the checker cannot match. That is the Lab 8 noise floor again: decide thresholds from repeated runs, not from one.

Each answer is also sent to Application Insights as an event called `rag_answer`. In the Azure portal, open **Application Insights**, then **Logs**, and run the query the script prints. Events take about 2 minutes to arrive.

```
customEvents
| where name == "rag_answer"
| extend run = tostring(customDimensions.run), fault = tostring(customDimensions.fault),
    top_score = todouble(customMeasurements.top_score),
    cost = todouble(customMeasurements.cost_usd),
    refused = tostring(customDimensions.refused)
| summarize answers = count(), mean_top_score = avg(top_score), total_cost_usd = sum(cost),
    refusals = countif(refused == "true") by run, fault
| order by run desc
```

8. Week 6: Privacy and capstone

Lab 11: Entitlements, PII and an auditable answer

Goal. Detect PII at ingestion, tag every chunk with sensitivity and region, resolve the filter on the server from the user's identity, filter before retrieval, write an audit record, and delete a customer from every store.

Databricks track

Open `lab11_privacy` in your copy of `freshcart_notebooks` and click Run all.

Run end to end on Azure Databricks on 24 September 2026. The lab code is the same file on both platforms, so the numbers in the command-line block below are what you should see.

Command-line track

```
python labs\lab11_privacy.py
```

What you should see:

```
rules found: [('email', 'priya.sharma@example.com'), ('aadhaar_like', '4111 1111 1111'),
('card_like', '4111 1111 1111 1111'), ('customer_id', 'CUS-40219'), ('context_number', 'card
4111')]

agent_south
  retrieved: ['refund_policy_v3_2024.pdf', 'support_sla_matrix.docx', ...]
  restricted chunks in context: 0 (must be 0)
supervisor
  retrieved: ['refund_claims_march_2024.xlsx', 'refund_claims_march_2024.xlsx', ...]

=== 5. erasure request for customer CUS-40219 ===
{'subject': 'CUS-40219', 'chunks_removed': 5, 'audit_records_removed': 1,
'cache_entries_removed': 1, 'verified_clean': True}
```

Look at the rules' mistakes as well as their catches: the start of the card number is also flagged as an Aadhaar-like number, the phone number with a space is missed, and the customer's name is not detected at all. The script ends by listing three places the deletion routine does not reach.

Azure track

```
python labs\lab11_privacy.py --azure
```

Azure AI Language runs the same tickets, and retrieval uses an Azure AI Search security filter built from the identity.

What you should see:

```

=== 1b. the same tickets through Azure AI Language PII ===
[('Person', 'Priya Sharma'), ('Email', 'priya.sharma@example.com'),
 ('PhoneNumber', '+91 98450 12345'), ('PersonType', 'Customer'),
 ('CreditCardNumber', '4111 1111 1111 1111')]
[('PersonType', 'caller'), ('PhoneNumber', '2345 6789 0123')]
service redaction: Ticket 5513: ***** gave PAN ABCDE1234F and Aadhaar ***** to
verify identity.

=== 2. entitlement tags on every chunk ===
sensitivity  region  chunks
restricted   -      2
restricted   North   5
restricted   South   4
standard     -     20

=== 3. same question, two users ===
agent_south (filter: superseded eq false and sensitivity ne 'restricted'
             and (region eq null or region eq 'South'))
  restricted chunks in context: 0 (must be 0)
supervisor (filter: superseded eq false)
  restricted chunks in context: 4 (allowed)
  answer: The customer who exceeded the rolling 30-day claim limit in March 2024 is
CUS-40219, ...

=== 5. erasure request for customer CUS-40219 ===
{'subject': 'CUS-40219', 'chunks_removed': 5, 'audit_records_removed': 1, ...,
'verified_clean': True}
Azure AI Search: deleted 5 documents, remaining mentions: 0

```

Compare the two detectors. The service finds the customer's name, which the rules miss. It also misses the PAN and the CUS- customer id, labels the Aadhaar number as a phone number, and keeps a closing bracket inside the phone number. Neither is enough alone: run both, and test both on your own tickets before trusting either.

Note: `--pgvector` adds row-level security inside PostgreSQL: the same SELECT returns different rows for the agent and the supervisor. It needs a PostgreSQL server with pgvector (trainer guide, section 10.5).

Lab 12: Capstone, one question through the whole system

Goal. Answer Priya's question from Day 1 end to end: "Customer 57 says the refund for a spilled biryani on 21 March 2024 was declined. Why, and who pays for the spillage?" Then show the evidence.

Databricks track

Open `lab12_capstone` in your copy of `freshcart_notebooks` and click Run all.

Run end to end on Azure Databricks on 24 September 2026. The lab code is the same file on both platforms, so the numbers in the command-line block below are what you should see.

Command-line track

```
python labs\lab12_capstone.py
```

What you should see:

```

=== 1. the number, from the governed warehouse ===
claim_date  order_id  status
2024-03-04   6985     Approved
2024-03-12   6986     Approved
2024-03-21   6987     Declined

=== 2-4. retrieval and answer for agent_south ===
retrieved: ['warehouse:fact_refund_claim', 'refund_policy_v2_2022.pdf', ...]
... A customer may make up to two refund claims in any rolling thirty day period. A
third ...

=== 5. quality gate on a deliberate regression (Lab 8, prompt change) ===
exit code 1: merge BLOCKED: 1 blocking gate(s) failed

=== evidence pack ===
agent_south  citations 4  restricted in context 0  faithfulness 1.0
supervisor   citations 4  restricted in context 3  faithfulness 1.0

```

The story resolves only if every week's work holds. The claim was on 21 March 2024, so the **2022 policy applied** (the 2024 policy starts on 1 April): two claims already in 30 days, so the third was declined. The agent never sees the partner agreement, which is restricted; the supervisor does, and learns the restaurant partner pays when packaging fails. The evidence pack is `out\capstone_evidence.json`.

Azure track

```
python labs\lab12_capstone.py --azure
```

What you should see:

```

=== 2-4. retrieval and answer for agent_south ===
retrieved: ['warehouse:fact_refund_claim', 'refund_policy_v2_2022.pdf',
'refund_policy_v2_2022.pdf', ...]
answer: Customer 57's refund claim for the spilled biryani on 21 March 2024 was declined
because
it was their third claim within a rolling thirty-day period. According to the refund
policy, a
customer may make up to two refund claims in any rolling thirty-day period, and a third and
any
subsequent claim within that period is automatically declined [0c9a4a7b6b8f],
[2f62bd7d3eae]. ...
citations verified: ['0c9a4a7b6b8f', '2f62bd7d3eae', '8b8249c86be7']

=== 2-4. retrieval and answer for supervisor ===
retrieved: ['warehouse:fact_refund_claim', 'refund_claims_march_2024.xlsx',
'partner_agreement_extract.pdf', ...]
answer: ... Regarding who pays for the spillage, since the spillage is related to packaging
inadequate for transport of liquid contents, the Partner bears the full cost of the refund
according to the FreshCart Restaurant Partner Agreement Schedule 4 ... [b0563f97ea12].

=== 5. quality gate on a deliberate regression (Lab 8, prompt change) ===
exit code 1: merge BLOCKED: 1 blocking gate(s) failed

=== evidence pack ===
agent_south citations 3 restricted in context 0 faithfulness 1.0 latency 12713.1 ms
supervisor  citations 3 restricted in context 3 faithfulness 1.0 latency 5763.7 ms
Azure OpenAI usage: 4 chat calls, 3454 tokens in, 624 out, about $0.0024

```

Both answers explain the decline from the warehouse facts and the 2022 policy. Only the supervisor's answer says who pays, because only the supervisor may see the partner agreement. The agent's answer falls back to the customer policy and never names the partner. Same question, same system, different entitlements.

The quality gate in step 5 runs the command-line version of Lab 8 on purpose: it is fast and gives the same result every time, which is what a demo in front of a panel needs.

The panel asks three questions, published in advance: How do you know? Show me what was retrieved. What would you fix first?

9. Troubleshooting

Every row below happened during testing or is the most common classroom failure. Ask for help after 5 minutes; do not debug an environment problem quietly for an hour.

What you see	Why	What to do
'py' is not recognized or Python opens the Microsoft Store	Python is not installed, or Windows' Store shortcut answers first	Install Python (appendix B.2). Use <code>py</code> , not <code>python</code> , until the environment is active
<code>ModuleNotFoundError: No module named 'numpy' (or sklearn, openai)</code>	The virtual environment is not active in this window	Run the two lines in appendix B.5. The prompt must start with <code>(.venv)</code>
The setup script shows nothing for several minutes at step 3	pip is downloading about 400 MB of packages quietly	Wait. On the lab machine the whole setup takes 7 to 8 minutes
The first lab run takes 30 to 40 seconds	Windows scans newly installed packages the first time they load	Normal. The next run takes about 10 seconds
Matplotlib is building the font cache (Lab 6)	First use of the chart library	Normal, once only
The notebook asks for a kernel, or <code>ModuleNotFoundError</code> inside a notebook cell	VS Code is using a different Python than the lab environment	Top right, Select Kernel, Python Environments , pick the one ending in <code>.venv</code>
Scanned PDF returns 0 characters	The file has no text layer. This is the Lab 1 lesson, not a bug	The Azure track OCRs it; the command-line track quarantines it
No recorded model output (Lab 5)	<code>data\golden\nl2sql_offline_replay.json</code> is missing	Download the lab pack again, or run the Azure track once with <code>--azure --record</code>
The Azure track needs these settings, but they are empty	<code>lab.env</code> is missing or in the wrong folder	Put <code>lab.env</code> next to <code>README.md</code> in the lab folder (appendix B.6)
Your Windows user name is 'admin', which every lab machine may share	No student id set	<code>setx STUDENT_ID yourname</code> and <code>set STUDENT_ID=yourname</code>
401 or Access denied due to invalid subscription key	Keys were rotated, or <code>lab.env</code> was edited by hand	Get a fresh <code>lab.env</code> from the trainer
429 or Rate limit	Many students called the model at the same second	The scripts retry automatically. If it persists, the trainer raises the deployment capacity (trainer guide, section 10.6)
PostgreSQL + pgvector (optional) FAIL	No PostgreSQL server is set up for the class	Ignore unless your trainer asks you to run <code>--pgvector</code>
The API in Lab 4 says the address is already in use	A server from an earlier try is still running	Close that window, or use <code>serve --port 8001</code> and ask <code>--port 8001</code>
Two pairs get different numbers on the Azure track	Live models vary a little between runs	Expected. Compare configurations; Lab 8 measures this spread on purpose
Ctrl+V does nothing in the lab machine	The browser tab does not have keyboard focus	Click inside the Command Prompt window, then paste

Appendix A. Command cheat sheet

Task	Command
Go to the lab folder and activate	<code>cd /d %USERPROFILE%\freshcart-labs then .venv\Scripts\activate</code>
Check the environment	<code>python setup\check_env.py (add --azure for Azure)</code>
Run a lab on the command-line track	<code>python labs\lab03_hybrid.py</code>
Run the same lab on Azure	<code>python labs\lab03_hybrid.py --azure</code>
See a script's options	Open the file in VS Code: <code>code labs\lab09_pipeline.py</code> . The top comment lists every option
Show the last exit code	<code>echo %ERRORLEVEL%</code>
Set your student id	<code>setx STUDENT_ID yourname and set STUDENT_ID=yourname</code>
Save output to a file	<code>python labs\lab07_lineage.py > out\lab07.txt</code>
Rebuild the warehouse	<code>python build_warehouse.py</code>
Run every lab once (trainer)	<code>python setup\smoke_test.py (add --azure)</code>

Appendix B. Running the labs on a Windows machine

The course runs on Azure Databricks, and section 2 is all you need for that. This appendix is the older path: the same twelve labs on a Windows machine with Python installed. Use it if you want to work offline, or if your trainer asks you to.

Time: about 15 minutes, most of it waiting for packages to download. Do this before Day 1.

B.1 Open the lab machine and a Command Prompt

1. Open the lab portal link from NovelVista in Chrome and sign in. Your Windows desktop opens in the browser tab.
2. Inside the lab machine, press the **Windows key + R**, type `cmd` and press Enter. A black Command Prompt window opens.
3. Optional but recommended: maximise the window so long output does not wrap.

Note: Keyboard shortcuts such as Ctrl+V go to the lab machine only while the browser tab has focus. If paste does not work, click inside the Command Prompt window first.

B.2 Check that Python is installed

```
py -0p
```

What you should see (the version number may differ):

```
-V:3.14 * C:\Users\admin\AppData\Local\Programs\Python\Python314\python.exe
```

Any version 3.11 or newer works. If the command says `py` is not recognised, or lists nothing, install Python 3.12 with one command, then close and reopen Command Prompt:

```
winget install -e --id Python.Python.3.12 --scope user
```

Watch out: Typing `python` on a fresh Windows machine can open the Microsoft Store instead of Python. That is a shortcut Windows installs, not Python. Use `py` until the virtual environment is active (appendix B.5); after that, `python` is correct.

B.3 Get the lab pack

Your trainer shares a download link in the class chat. Replace `<LINK>` with it, keeping the double quotes:

```
cd /d %USERPROFILE%
curl -L -o freshcart-labs.zip "<LINK>"
tar -xf freshcart-labs.zip
cd freshcart-labs
```

`curl` and `tar` are built into Windows 10 and 11, so nothing else needs installing. The folder is now `C:\Users\<you>\freshcart-labs`.

B.4 Run the setup script

```
setup\setup_windows.cmd
```

This creates a private Python environment in the `.venv` folder, installs the packages, builds the warehouse and checks everything. It also installs what the last step of labs 1, 6, 9 and 10 needs, which is the largest part of the download. The script took 7 to 8 minutes on the lab machine before that was added, so allow more; the install step prints nothing for several minutes, which is normal.

What you should see at the end:

```
[4/5] Building the warehouse (data\freshcart.db)...
dim_customer          202 rows
dim_restaurant        40 rows
orders                3866 rows
orders_v2             5988 rows
order_items           14198 rows
fact_refund_claim     277 rows
daily_customer_snapshot 2400 rows

[5/5] Checking the environment...
PASS Python 3.11+      0.0s Python 3.14.7 (C:\Users\admin\freshcart-
labs\.venv\Scripts\python.exe)
PASS core packages    37.6s numpy 2.5.3, sklearn 1.9.1, pypdf 6.18.1, docx 1.2.0, pptx
1.0.2, ...
PASS lab packages     2.8s matplotlib 3.11.2, fastapi 0.141.1, uvicorn 0.53.0
PASS lab data         0.0s 7 corpus files, warehouse 904 KB
PASS Lab 1 smoke test 0.2s 17 units, 30 chunks, empty (expected: the scanned SOP): [...]
PASS warehouse query  0.0s orders_v2 has 5988 rows

6/6 checks passed
```

Note: A current run prints one more line, tool step packages, and ends with 7/7 checks passed. That line covers the packages the last step of labs 1, 6, 9 and 10 needs, including dbt.

Note: The first check takes about 40 seconds the first time because Windows scans the new packages. Later runs take a few seconds.

B.5 Every time you open a new Command Prompt

Run these two lines first. The prompt then starts with (.venv), which means the lab environment is active.

```
cd /d %USERPROFILE%\freshcart-labs
.venv\Scripts\activate
```

B.6 Add the Azure track

Do this when your trainer says the Azure track is open. You need the file `lab.env` from your trainer.

1. Save `lab.env` into the lab folder, next to `README.md`. If the trainer shares a link, use: `curl -L -o lab.env "<LINK>"` from inside the lab folder.
2. Choose a student id: short, lowercase, letters and digits only, for example your first name and a number. Set it for this window and for future windows:

```
setx STUDENT_ID priya07
set STUDENT_ID=priya07
```

1. Check that every Azure service answers:

```
python setup\check_env.py --azure
```

What you should see (the PostgreSQL line may say FAIL; it is optional):

```
PASS Azure track packages 6.4s openai 3.14.0, requests 2.34.2, psycopg 3.3.5
PASS lab.env present      0.0s True
PASS Azure OpenAI         6.4s embedding dim 1536, chat says 'ready'
PASS Azure AI Search      0.4s indexes used 0 of 3
PASS Azure AI Language    1.2s PII entities found: ['Person', 'Email']
PASS Application Insights 0.4s test event accepted
FAIL PostgreSQL + pgvector (optional) ConnectionTimeout: connection timeout expired

12/13 checks passed, 1 optional check not available (fine unless your trainer asked for it)
```

Watch out: `lab.env` contains keys that spend course money. Do not email it, paste it into chat or commit it to Git. The trainer rotates the keys after the course, which makes every copy useless.

Watch out: Every lab machine may have the same Windows user name (`admin`). If you skip the student id, the Azure track stops with a message asking you to set one. This protects other students' work, not just yours.